

INTELLIGENT DOCUMENT QUESTION-ANSWERING ASSISTANT

Using Retrieval-Augmented Generation (RAG)

Professional Academic & Technical Project Report

Prepared By

M.ARULKUMAR

Artificial Intelligence / Machine Learning / Generative AI

CONTENT

1. Abstact.....	3
2. Introduction.....	3
3. Problem Statement.....	4
4. Objectives.....	4
5. System Architecture.....	4
6. Technology stack.....	5
7. Document Ingestion.....	5
8. Text Extraction.....	5
9. Text Preprocessing.....	5
10. Text Chunking.....	6
11. Embedding Generation.....	6
12. FAISS Vector Database.....	6
13. Question Processing.....	6
14. Similarity search.....	6
15. LLM-Based Answer Generation.....	7
16. Frontend and Backend Communication.....	7
17. User Interface.....	7
18. Key Features.....	7
19. Security Consideration.....	8
20. Current Limitations.....	8
21. Future Enhancements.....	8
22. Expected Outcome.....	8
23. Conclusion.....	9

1. Abstract

The Intelligent Document Question-Answering Assistant is an AI-powered application designed to allow users to interact with their documents through natural-language questions. The system supports document inputs such as PDF and TXT files, as well as copied and pasted text.

The project implements a Retrieval-Augmented Generation (RAG) architecture. Uploaded document content is extracted, cleaned, divided into meaningful chunks, converted into vector embeddings using a Gemini embedding model, and stored in a FAISS vector database. When a user asks a question, the system converts the question into an embedding and performs similarity search to retrieve the most relevant document sections. These retrieved sections are then provided to a Gemini Large Language Model (LLM), which generates a context-grounded answer.

The combination of FastAPI, Gemini Embeddings, FAISS, LangChain Text Splitters, PyMuPDF, and a web-based frontend provides a practical foundation for a production-oriented document intelligence system.

2. Introduction

Traditional document-reading systems require users to manually search through large amounts of information. This becomes inefficient when documents contain hundreds of pages or complex technical information.

The proposed Document Q&A Assistant addresses this problem by providing a conversational interface for document analysis. Instead of manually searching a document, users can simply ask questions in natural language. The system retrieves relevant content from the document and generates an answer based on the available information.

The project follows the principle of Retrieval-Augmented Generation, where document retrieval is performed before LLM-based answer generation.

3. Problem Statement

Users often face difficulties when extracting specific information from large documents because manual document searching is time-consuming, keyword-based search may fail to understand semantic meaning, large documents may contain redundant information, general-purpose LLMs may generate information that is not present in the document, and processing the entire document for every question can be inefficient.

Therefore, there is a need for an intelligent system that can retrieve relevant document information and generate accurate, context-aware answers.

4. Objectives

1. Develop an AI-powered document question-answering system.
2. Support PDF and TXT document ingestion.
3. Support copied and pasted document content.
4. Extract and preprocess document text.
5. Divide documents into manageable chunks.
6. Generate semantic embeddings for document chunks.
7. Store embeddings efficiently using FAISS.
8. Perform similarity-based document retrieval.
9. Use an LLM for context-grounded answer generation.
10. Provide an interactive web-based user interface.
11. Reduce the possibility of unsupported or hallucinated answers.

5. System Architecture

The overall architecture follows this pipeline:

User → Web Frontend → FastAPI Backend → Text Extraction → Text Cleaning → Chunking → Gemini Embeddings → FAISS Vector Database → User Question → Question Embedding → Similarity Search → Relevant Document Chunks → Gemini LLM → Generated Answer → Frontend.

6. Technology Stack

Component	Technology
Frontend	HTML, CSS, JavaScript
Backend	FastAPI
Programming Language	Python
PDF Processing	PyMuPDF
Text Chunking	LangChain RecursiveCharacterTextSplitter
Embedding	Gemini Embedding Model
Vector Database	FAISS
Numerical Processing	NumPy
LLM	Gemini
Configuration	python-dotenv
API Communication	REST API

7. Document Ingestion

7.1 File Upload

Users can upload PDF and TXT documents. The backend validates file extension, file size, empty files, and readable document content. The current implementation applies a 10 MB file-size limit.

7.2 Copy and Paste

Users can directly paste document content into the application. The content is sent to the FastAPI backend for processing, allowing document analysis even when the original file is not available.

8. Text Extraction

For PDF documents, the project uses PyMuPDF to extract text page by page. The extracted content is combined into a single text representation while maintaining page information. TXT documents are read using Python file handling functionality.

9. Text Preprocessing

Before creating embeddings, the extracted content is cleaned. The preprocessing approach removes unnecessary line breaks, normalizes whitespace, removes unnecessary spaces, and produces clean document text for subsequent chunking.

10. Text Chunking

Large documents cannot efficiently be processed as a single embedding. Therefore, the project uses RecursiveCharacterTextSplitter from LangChain.

Current configuration:

- Chunk size: 1000 characters
- Chunk overlap: 200 characters

The overlap helps preserve contextual information between neighboring chunks.

11. Embedding Generation

After chunking, each document chunk is converted into a numerical vector using a Gemini embedding model. These vectors represent the semantic meaning of the corresponding text chunks and enable semantic retrieval rather than simple keyword matching.

12. FAISS Vector Database

The generated embeddings are stored in FAISS (Facebook AI Similarity Search). The project uses an IndexFlatIP index for similarity comparison. The vector database enables similarity-based retrieval between the user's question and stored document chunks.

13. Question Processing

When the user asks a question, the question itself is converted into an embedding using the same embedding model. The resulting question vector is compared with the vectors stored in FAISS.

14. Similarity Search

The system performs similarity search to identify the most relevant document chunks. The current implementation retrieves the top 3 relevant chunks. Each result contains the relevant document content and a similarity score.

15. LLM-Based Answer Generation

After retrieving relevant document chunks, they are combined into a context. The context and user question are then provided to the Gemini LLM. The system instructs the model to answer using only the retrieved document context. This approach reduces the likelihood of unsupported information being generated.

16. Frontend and Backend Communication

The frontend communicates with the FastAPI backend through REST API endpoints.

Important endpoints:

- POST /upload — Upload and process PDF/TXT documents.
- POST /content — Process copied or pasted document content.
- POST /ask — Process user questions and return the generated answer.

The generated LLM response is displayed in the frontend answer area using the HTML element `<p id="result"></p>`.

17. User Interface

The frontend provides a simple document interaction interface containing a document upload button, text input area, question input, send button, uploaded filename display, answer display area, and notification system.

18. Key Features

- PDF and TXT document support.
- Copy/paste document support.
- Semantic search using embeddings.
- FAISS vector storage and retrieval.
- Context-grounded LLM generation.
- Input and file validation.
- FastAPI REST API architecture.
- Modular document-processing pipeline.

19. Security Considerations

The application uses environment variables for API credentials. The Gemini API key should not be hard-coded into source code or uploaded to GitHub. The .env file, virtual environment, cache files, and uploaded documents should be excluded using .gitignore.

20. Current Limitations

- Retrieval quality depends on chunk size and embedding quality.
- OCR is required for scanned or image-only PDFs.
- FAISS data is currently maintained in application memory.
- The system retrieves a fixed number of chunks.
- Advanced reranking is not yet implemented.
- Conversation memory is not currently included.
- Authentication and user-level document isolation are not yet implemented.
- Production deployment requires additional infrastructure and configuration.

21. Future Enhancements

- Query rewriting and improved retrieval.
- Hybrid search and advanced reranking.
- Persistent vector database.
- Docker containerization and cloud deployment.
- Authentication and user-specific document collections.
- Document metadata management.
- Groundedness evaluation and citation generation.
- Conversation memory.
- Multi-document question answering.
- Agent-based workflows using LangGraph.

22. Expected Outcome

The expected outcome is a functional AI-powered document assistant capable of accepting user documents, extracting content, creating meaningful chunks, generating semantic embeddings, storing vectors in FAISS, retrieving relevant information, generating document-grounded answers using an LLM, and displaying the generated response through a web interface.

23. Conclusion

The Intelligent Document Question-Answering Assistant demonstrates the practical application of Generative AI, embeddings, vector databases, semantic search, and Retrieval-Augmented Generation.

By combining FastAPI with document processing, Gemini embeddings, FAISS similarity search, and Gemini LLM-based generation, the system creates a complete pipeline from document ingestion to intelligent question answering. The project provides a strong academic and portfolio foundation and can be extended into a production-style RAG application with persistent storage, authentication, advanced retrieval, reranking, citations, evaluation, and cloud deployment.